Construction Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Hardik Baser	EN22CS301380
2	Hariom Kale	EN22CS301381
3	Harshita Parmar	EN22CS301408
4	Hricha Laad	EN22CS301429
5	Huzefa Burhani	EN22CS301432

Group Name:Group02D4

Project Number:AAI-38

Industry Mentor Name:

University Mentor Name: Prof. Pradeep Baniya

Academic Year:2025-2026

1. Introduction

1.1 Scope of the Document

This document provides a High-Level Design (HLD) for the **Construction Planning Assistant**, an AI-powered system that transforms high-level construction goals into structured plans and schedules.

It covers:

- System architecture
- Data flow and components

- APIs and interfaces
- Non-functional requirements

1.2 Intended Audience

This document is intended for:

- Developers and Software Engineers
- System Architects
- Project Managers
- AI/ML Engineers
- Stakeholders involved in system design and deployment

1.3 System Overview

The system is an **Agentic AI-based application** that:

- Takes construction goals as input
- Uses **Planner Agent** to generate plans
- Uses **Scheduler Agent** to create timelines
- Enhances output using **RAG (Retrieval Augmented Generation)**

2. System Design

2.1 Application Design

The system follows a **layered architecture**:

Layers:

1. Presentation Layer (Frontend)
2. Application Layer (FastAPI Backend)

3. AI Agent Layer
4. Data Layer (Vector DB + External Data)

2.2 Process Flow

1. User enters construction goal
2. Request sent to FastAPI backend
3. Planner Agent generates structured plan
4. RAG fetches relevant knowledge
5. Scheduler Agent creates timeline
6. Final output returned to user

2.3 Information Flow

- Input: User goals
- Processing: AI agents + RAG
- Output:
    - Construction Plan
    - Timeline/Schedule

Flow:
User → Frontend → FastAPI → Planner Agent → RAG → Scheduler → Response

2.4 Components Design

Frontend

- Input forms for goals
- Displays plans and schedules

Backend (FastAPI)

- API handling
- Request routing
- Agent orchestration

Planner Agent

- Converts goals → structured plan

Scheduler Agent

- Converts plan → timeline

RAG Module

- Retrieves relevant construction knowledge

Vector Database (ChromaDB)

- Stores embeddings for context retrieval

2.5 Key Design Considerations

- Scalability (multi-user support)
- Modularity (agent-based system)
- Performance (fast inference using Groq)
- Accuracy (RAG-based context awareness)
- Extensibility (add more agents easily)

2.6 API Catalogue

API	Method	Description
/generate-plan	POST	Generates construction plan
/generate-schedule	POST	Generates timeline
/get-data	GET	Fetch stored data
/health	GET	System health check

3. Data Design

3.1 Data Model

Entities:

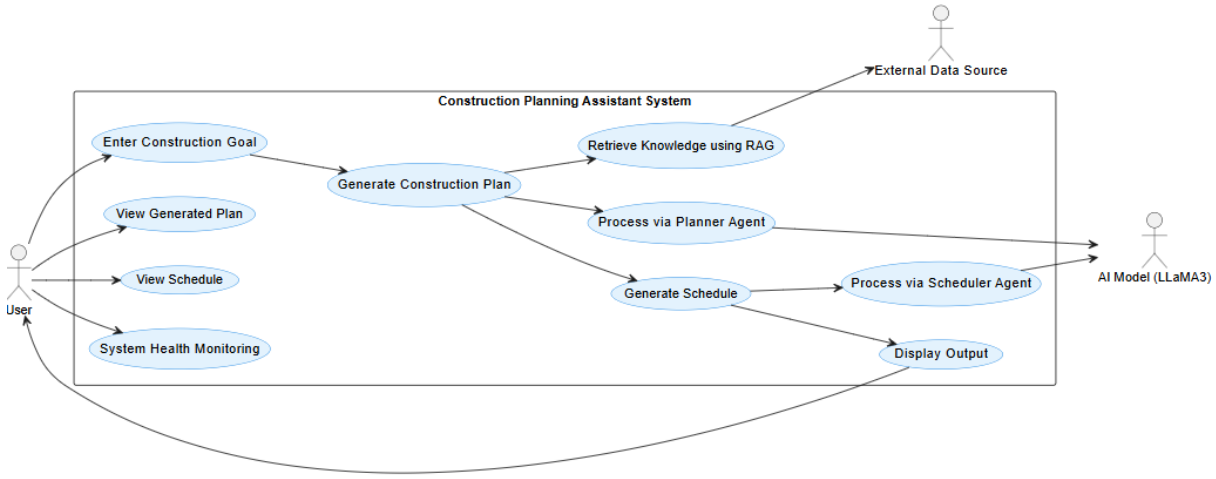
- User Input (Goal)
- Plan Output
- Schedule Output
- Knowledge Data (RAG)

Use Case Diagram

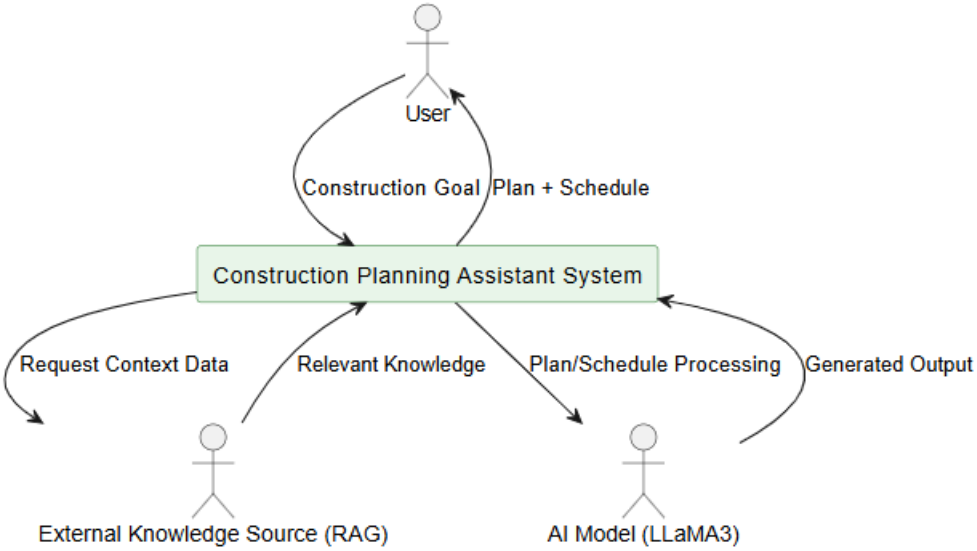
The Use Case Diagram represents how different actors (users and external systems) interact with the Construction Planning Assistant System.

It shows:

- What actions the system performs
- Who interacts with the system
- How the workflow moves from input → processing → output.



DFD Level 0 (Context Diagram) :



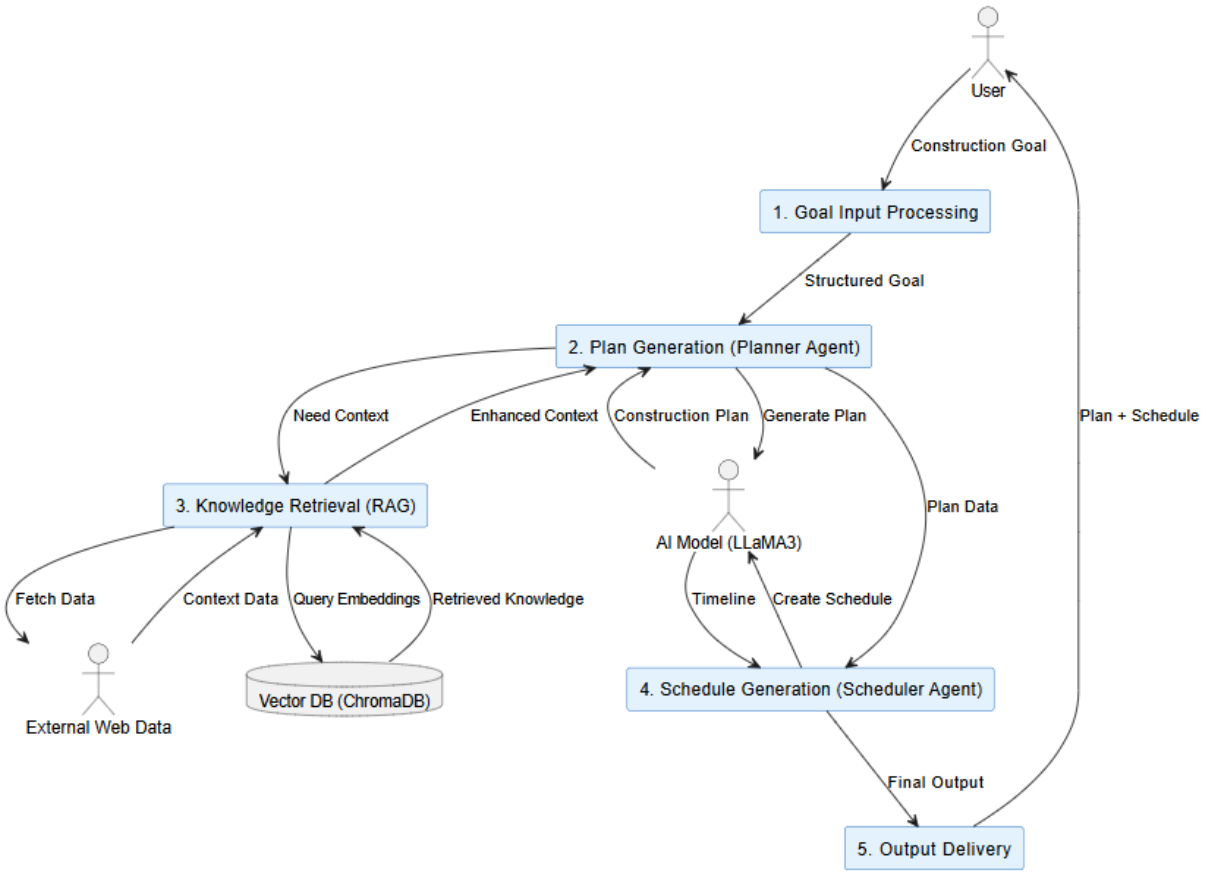
This diagram shows:

- Entire system as one process
- Input from user
-
- Output to user
- Interaction with AI and RAG

Flow:

1. User gives goal
2. System processes using AI
3. System fetches knowledge using RAG
4. Final plan & schedule returned

DFD Level 1 :



This diagram explains internal system workflow.

Goal Input Processing

- User provides construction goal
- System validates and structures input

Plan Generation

- Planner Agent creates detailed plan

Knowledge Retrieval (RAG)

- Fetches:
  - Construction rules
  - Material info
  - Best practices

Sources:

- Vector DB
- External web data

Schedule Generation

- Scheduler Agent creates timeline

Output Delivery

- Final results shown to user

3.2 Data Access Mechanism

- REST APIs for frontend-backend communication
- Vector search for RAG retrieval
- JSON-based data exchange

3.3 Data Retention Policies

- Temporary storage for session data
- Logs stored for debugging
- Optional long-term storage for plans

3.4 Data Migration

- Export/import via JSON
- Backup from vector DB
- Versioning for model updates

4. Interfaces

User Interface:

- Web-based UI (HTML/CSS/JS)

System Interfaces:

- FastAPI REST APIs
- AI model APIs (Groq / LLaMA3)

5. State and Session Management

- Stateless backend (REST-based)
- Session handled via:
  - Tokens or session IDs
- Temporary user context maintained

6. Caching

- Frequently used queries cached
- RAG results cached for faster retrieval
- Improves response time

7. Non-Functional Requirements

7.1 Security Aspects

- Input validation
- API authentication
- Data encryption (HTTPS)
- Secure model access

7.2 Performance Aspects

- Low latency AI inference (Groq)
- Efficient vector search
- Scalable backend (FastAPI async support)

8. References

- LangChain Documentation

- FastAPI Documentation
- CrewAI Framework
- ChromaDB Documentation
- Groq LLaMA3 API Docs